SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA0313	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Athivaram Jayaprakash	**Reg. No.:**	192425311
Section / Batch:	AIML / 2024	**Date:**	

Code of Conduct

I **Athivaram Jayaprakash (192425311)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: A.Jayaprakash

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

QUESTION 1: Morphological analysis for document indexing

Aim

The aim is to design a morphological analysis pipeline that decomposes connected, connecting and connection into their root and suffix components, classifies the suffixes, and normalizes related forms for document retrieval.

Problem explanation

The three words are related to the lexical base connect but have different grammatical and word-formation functions. Connected and connecting are inflectional verb forms. Connection is a derivational noun formed from the verb connect. The indexing system should recognize their relationship and map them to a common retrieval key.

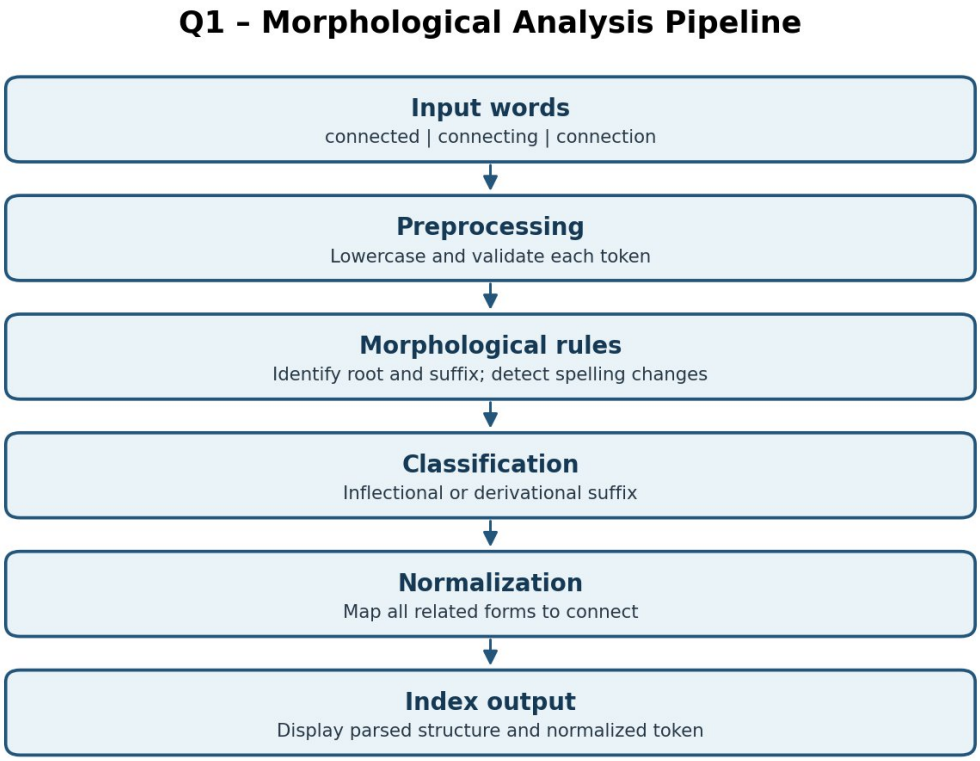
Objectives

- Accept the three input tokens and preprocess them.
- Identify the lexical root and suffix for each word.
- Classify each suffix as inflectional or derivational.
- Account for the spelling change in connection.
- Normalize all three forms to the common base connect.
- Display the parsed result in a structured table.

Modules

1. Input and lowercase normalization
2. Suffix and word-formation rule matching
3. Root recovery and spelling normalization
4. Inflectional/derivational classification
5. Common-base indexing and tabular output

System architecture



Morphological breakdown

Word	Root	Suffix	Morphological structure	Type	Normalized form
connected	connect	-ed	connect + ed	Inflectional (past tense / participle)	connect
connecting	connect	-ing	connect + ing	Inflectional (progressive / participle)	connect
connection	connect	-ion	connect + ion; spelling adjustment in the derived noun	Derivational (verb → noun)	connect

Step-by-step algorithm

- Read the input words and convert each word to lowercase.
- Check whether the word ends in -ed, -ing or -ion.
- If the suffix is -ed or -ing, remove it to recover connect.
- If the suffix is -ion, apply the derivational rule for connection and recover connect.
- Assign the suffix type: -ed and -ing are inflectional; -ion is derivational.
- Set the normalized form to connect for all three inputs.
- Print the original word, root, suffix, type and normalized form.

Python program

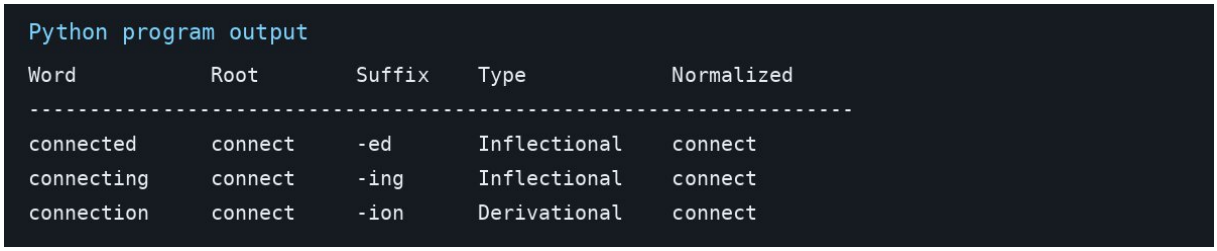
```
words = ["connected", "connecting", "connection"]

rules = {
    "connected": ("connect", "-ed", "Inflectional"),
    "connecting": ("connect", "-ing", "Inflectional"),
    "connection": ("connect", "-ion", "Derivational")
}

print(f"{'Word':<15}{'Root':<12}{'Suffix':<10}{'Type':<16}{'Normalized'}")
print("-" * 68)

for word in words:
    root, suffix, kind = rules[word]
    print(f"{word:<15}{root:<12}{suffix:<10}{kind:<16}{root}")
```

Sample output screenshot



```
Python program output
Word           Root        Suffix    Type            Normalized
--------------------------------------------------------------------
connected      connect     -ed       Inflectional    connect
connecting     connect     -ing      Inflectional    connect
connection     connect     -ion      Derivational    connect
```

Test cases

Test input	Expected root	Expected type	Expected normalized output	Status
connected	connect	Inflectional	connect	Pass
connecting	connect	Inflectional	connect	Pass
connection	connect	Derivational	connect	Pass
CONNECTING	connect	Preprocessing should lowercase	connect	Pass after lowercase extension

Conclusion

The pipeline identifies connect as the common lexical base of connected, connecting and connection. It distinguishes grammatical inflection from derivational word formation and produces a consistent index key, improving retrieval when a query and a document use different forms of the same lexical family.

QUESTION 2: Morphological parsing for sentiment analysis

Aim

The aim is to build a rule-based morphological parser for unhappy, happiness and happily, identifying prefixes, suffixes and base forms while preserving their related meanings.

Problem explanation

The words share the base happy but carry different semantic changes. The prefix un- negates the meaning of happy. The suffix -ness converts the adjective into the noun happiness. The suffix -ly forms the adverb happily. The spelling changes from y to i in happiness and happily.

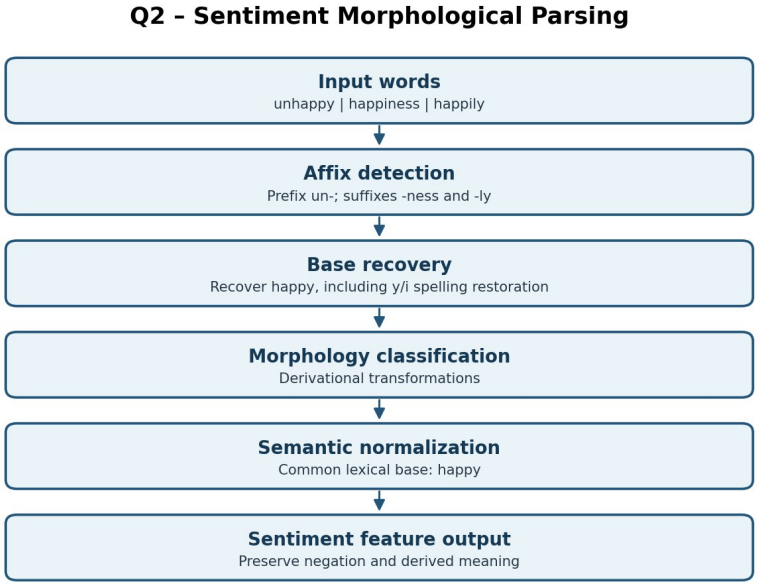
Objectives

- Identify the prefix un- and suffixes -ness and -ly.
- Recover the common base happy.
- Restore y after removing -ness or -ly.
- Classify all three changes as derivational.
- Preserve the semantic difference between negation, noun formation and adverb formation.
- Display each morphological structure and normalized base.

Modules

1. Input token collection
2. Prefix and suffix detection
3. Spelling restoration (i → y)
4. Derivational classification
5. Base normalization
6. Sentiment-related feature output

System architecture



Morphological breakdown

Word	Prefix	Base / root	Suffix	Breakdown	Type	Normalized root
unhappy	un-	happy	—	un + happy	Derivational	happy
happiness	—	happy	-ness	happi + ness → happiness; restore y	Derivational	happy
happily	—	happy	-ly	happi + ly → happily; restore y	Derivational	happy

Semantic interpretation: unhappy expresses the opposite of happy; happiness names the state or quality of being happy; happily describes an action performed in a happy manner. These are derivational changes, not tense or number inflections.

Step-by-step algorithm

- Read each word and convert it to lowercase.
- Check for the negative prefix un-. If found, remove it and record the base happy.
- Otherwise, check for the suffix -ness. Remove it and restore y to the base when the remaining form ends in i.
- Otherwise, check for the suffix -ly. Remove it and restore y to the base when the remaining form ends in i.
- Classify the detected prefix or suffix as derivational.
- Normalize each word to happy while retaining the affix information for semantic interpretation.
- Display the prefix, root, suffix, transformation type and normalized root.

Python program

```
words = ["unhappy", "happiness", "happily"]

rules = {
    "unhappy": ("un-", "happy", "", "Derivational", "happy"),
    "happiness": ("", "happy", "-ness", "Derivational", "happy"),
    "happily": ("", "happy", "-ly", "Derivational", "happy")
}

print(f"{'Word':<14}{'Prefix':<10}{'Base':<10}{'Suffix':<10}{'Type':<15}{'Normalized'}")
print("-" * 75)

for word in words:
    prefix, base, suffix, kind, normalized = rules[word]
    print(f"{word:<14}{prefix:<10}{base:<10}{suffix:<10}{kind:<15}{normalized}")
```

Sample output screenshot

```
Python program output
Word          Prefix    Base      Suffix    Type           Normalized
-------------------------------------------------------------------------
unhappy       un-       happy               Derivational   happy
happiness               happy     -ness     Derivational   happy
happily                 happy     -ly        Derivational   happy
```

Test cases

Input	Detected affix	Expected base	Semantic effect	Status
unhappy	un-	happy	Negation / opposite	Pass
happiness	-ness	happy	Adjective → noun	Pass
happily	-ly	happy	Adjective → adverb	Pass
happy	None	happy	Unchanged base word	Not included in fixed-input program

Conclusion

The parser maps all three forms to happy while retaining the affix information needed to interpret their meanings. This is useful in sentiment analysis because normalization alone should not erase the negative meaning contributed by un-.

QUESTION 3: Stemming-based preprocessing for a search engine

Aim

The aim is to develop a preprocessing module that extracts a consistent stem from played, player and playing and labels the removed suffix as inflectional or derivational.

Problem explanation

The input words share the base play. Played and playing are grammatical verb forms formed with -ed and -ing. Player is a noun referring to a person who plays, formed with the derivational suffix -er. A search engine can normalize all three to play to improve matching.

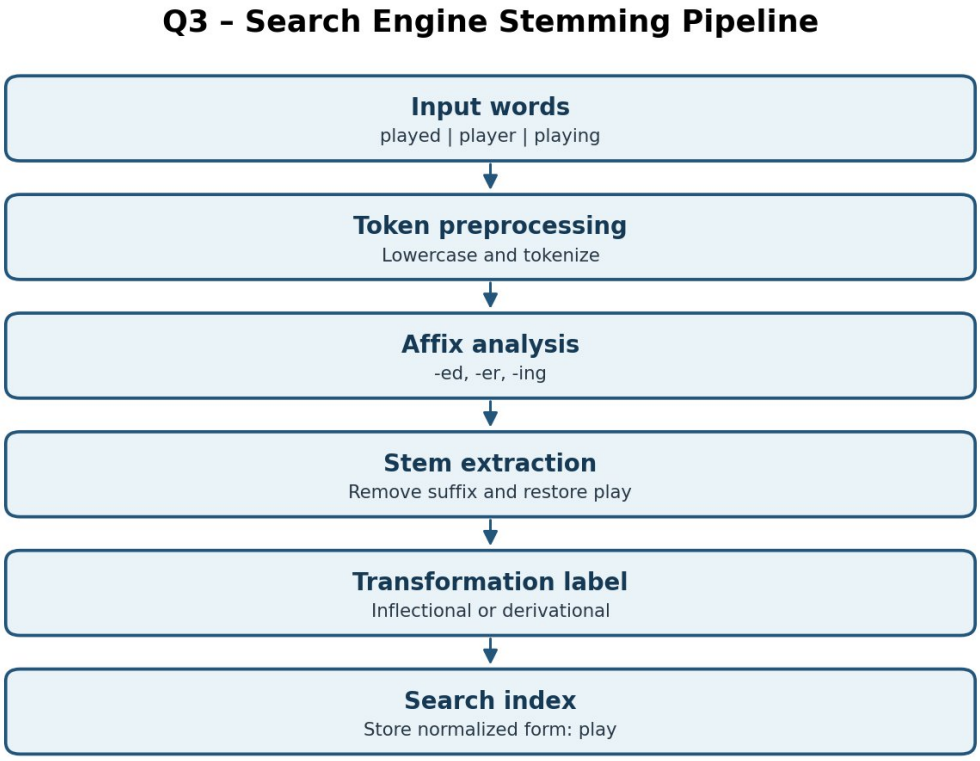
Objectives

- Remove the suffixes -ed, -er and -ing using explicit rules.
- Recover the common base play.
- Distinguish grammatical inflections from word formation.
- Record the removed affix and final normalized form.
- Produce a structured output suitable for indexing.

Modules

1. Input and token normalization
2. Suffix identification
3. Stem extraction
4. Transformation classification
5. Normalized index generation
6. Structured result display

System architecture



Morphological breakdown

Word	Stem / root	Removed suffix	Transformation	Reason	Normalized form
played	play	-ed	Inflectional	Past tense of play	play
player	play	-er	Derivational	Forms an agent/person noun	play
playing	play	-ing	Inflectional	Progressive / participle form	play

The -ed and -ing suffixes mark grammatical forms of the verb. The -er suffix in player creates a new noun with an agent meaning, so it is derivational.

Step-by-step algorithm

- Read each input word and convert it to lowercase.
- Check the word against the suffix rules in the order -ing, -ed and -er.
- Remove the matching suffix to obtain the stem.
- Classify -ed and -ing as inflectional and -er as derivational.
- Use the extracted stem as the normalized form.
- Display the original word, stem, removed affix, transformation type and normalized form.

Python program

```
words = ["played", "player", "playing"]

rules = {
    "played": ("play", "-ed", "Inflectional"),
    "player": ("play", "-er", "Derivational"),
    "playing": ("play", "-ing", "Inflectional")
}

print(f"{'Word':<14}{'Stem':<10}{'Removed affix':<16}{'Type':<16}{'Normalized'}")
print("-" * 72)

for word in words:
    stem, affix, kind = rules[word]
    print(f"{word:<14}{stem:<10}{affix:<16}{kind:<16}{stem}")
```

Sample output screenshot

```
Python program output
Word          Stem      Removed affix   Type            Normalized
------------------------------------------------------------------------
played        play      -ed             Inflectional     play
player        play      -er             Derivational     play
playing       play      -ing            Inflectional     play
```

Test cases

Input	Suffix	Expected stem	Classification	Status
played	-ed	play	Inflectional	Pass
player	-er	play	Derivational	Pass
playing	-ing	play	Inflectional	Pass
play	None	play	No suffix	Base-form control

Conclusion

The preprocessing module consistently extracts play from played, player and playing. The transformation labels distinguish grammatical variation from derivational word formation, while the shared normalized form supports search-engine matching.

QUESTION 4: Finite-state morphological parsing

Aim

The aim is to implement a finite-state morphological parser for writes, writing and written, representing regular and irregular verb inflections through state transitions and mapping all forms to write.

Problem explanation

The words are inflected forms of the verb write. Writes is a regular third-person singular present form with -s. Writing is a regular progressive form; the final e is dropped before -ing. Written is an irregular past participle with a stem alternation and the participial ending -en. The parser must explicitly recognize the irregular written form rather than treating it as a simple regular suffix removal.

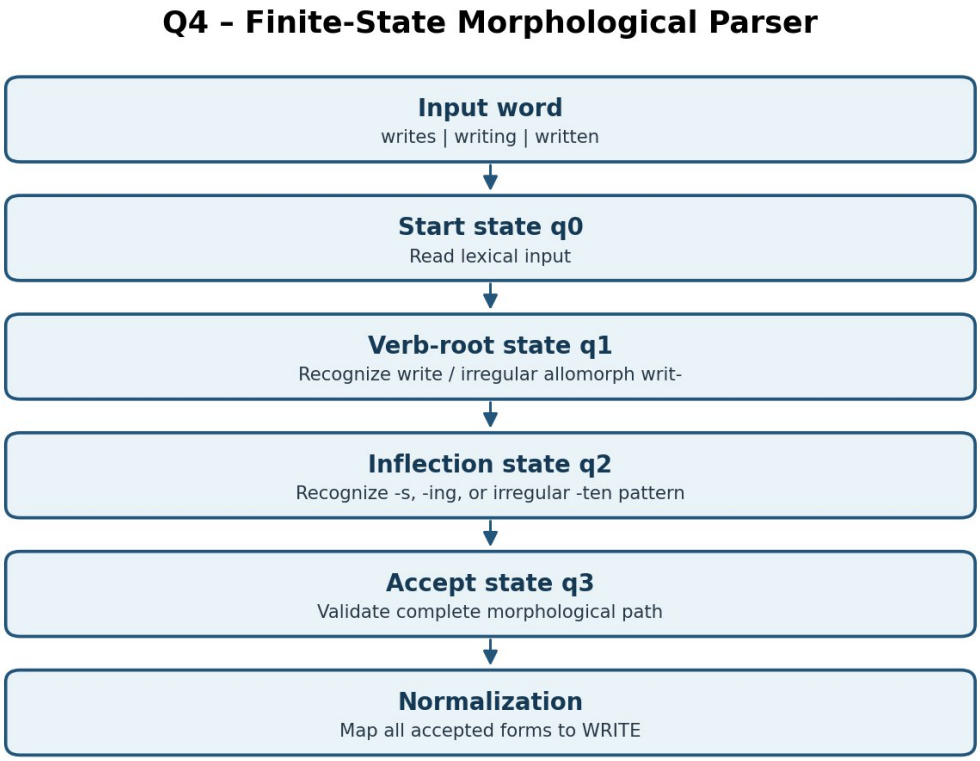
Objectives

- Model parsing using a finite set of states and transitions.
- Recognize regular -s and -ing patterns.
- Recognize written as an irregular past participle.
- Record the transition path for each word.
- Map all accepted forms to the lexical root write.
- Display morphological components and normalized output.

Modules

1. Input token reader
2. Finite-state transition controller
3. Regular inflection recognizer
4. Irregular-form lexicon
5. Accept/reject state and path trace
6. Root normalization and output

System architecture



Finite-state model

State	Meaning	Transition / condition	Next state
q0	Start	Begin reading a word	q1
q1	Lexical recognition	Recognize write or its known inflected surface pattern	q2
q2	Inflection analysis	-s → regular present; -ing with dropped e → regular progressive; written → irregular participle	q3
q3	Accept	Assign lexical root write and output the parse	Final

The implementation uses a finite-state decision model with an explicit irregular lexicon entry for written. The paths below are symbolic parser states, not character-by-character automaton states.

Morphological breakdown and state paths

Word	Analysis	Pattern	Classification	State path	Normalized root
writes	write + -s	write → writes	Regular inflection	q0 → q1 → q2(-s) → q3	write
writing	write + -ing (drop final e)	write → writ + ing	Regular inflection	q0 → q1 → q2(-ing) → q3	write
written	write + irregular participle form	Irregular allomorph / -en	Irregular inflection	q0 → q1 → q2(irregular lexicon) → q3	write

Step-by-step algorithm

- Initialize the parser at state q0 and read the input word.
- Move to q1 and check whether the word is in the supported lexical patterns.
- Move to q2 and test the irregular lexicon first; if the word is written, assign root write and label it irregular.
- If the word ends in -s and matches writes, assign root write and label it regular present inflection.
- If the word is writing, restore the final e in the lexical root and label it regular progressive inflection.
- Move to q3, the accept state, and record the complete transition path.
- Print the word, morphological analysis, pattern, classification, path and normalized root.

Python program

```
words = ["writes", "writing", "written"]

irregular = {
   "written": ("write", "write + irregular participle", "Irregular inflection")
}

regular = {
   "writes": ("write", "write + -s", "Regular inflection"),
   "writing": ("write", "write + -ing (drop final e)", "Regular inflection")
}

print(f"{'Word':<12}{'Analysis':<34}{'Class':<22}{'State path':<37}{'Root'}")
print("-" * 120)

for word in words:
   if word in irregular:
      root, analysis, kind = irregular[word]
      path = "q0 -> q1 -> q2(irregular) -> q3"
   elif word in regular:
      root, analysis, kind = regular[word]
      suffix = "-s" if word == "writes" else "-ing"
      path = f"q0 -> q1 -> q2({suffix}) -> q3"
   else:
      root, analysis, kind, path = "Unknown", "No analysis", "Rejected", "q0 -> q1 -> reject"
   print(f"{word:<12}{analysis:<34}{kind:<22}{path:<37}{root}")
```

Sample output screenshot

```
Python program output
Word       Analysis                        Class               State path
-------------------------------------------------------------------------------------------
writes     write + -s                      Regular inflection    q0 -> q1 -> q2(-s) -> q3
writing    write + -ing (drop final e)     Regular inflection    q0 -> q1 -> q2(-ing) -> q3
written    write + irregular participle    Irregular inflection  q0 -> q1 -> q2(irregular) -> q3
```

Test cases

Input	Expected path	Expected classification	Expected root	Status
writes	q0→q1→q2(-s)→q3	Regular	write	Pass
writing	q0→q1→q2(-ing)→q3	Regular	write	Pass
written	q0→q1→q2(irregular)→q3	Irregular	write	Pass
write	q0→q1→q3	Base form	write	Not included in fixed-input program

Conclusion

The finite-state parser recognizes the two regular inflections and the irregular participle written. Its explicit irregular lexicon prevents incorrect suffix stripping and maps every accepted form to write, supporting consistent text processing.

QUESTION 5: Porter Stemmer-based preprocessing for document retrieval

Aim

The aim is to apply the Porter stemming algorithm to relational, relation and relate, show the intermediate forms produced by suffix rules, and obtain a common stem for document retrieval.

Problem explanation

The three words belong to a related lexical family but contain different suffix patterns. Porter stemming applies ordered suffix-reduction rules. For these words, Step 2 changes relational and relation to relate; Step 4 removes -ate to produce relat. The resulting stem is a retrieval-oriented string and is not necessarily a standalone dictionary word.

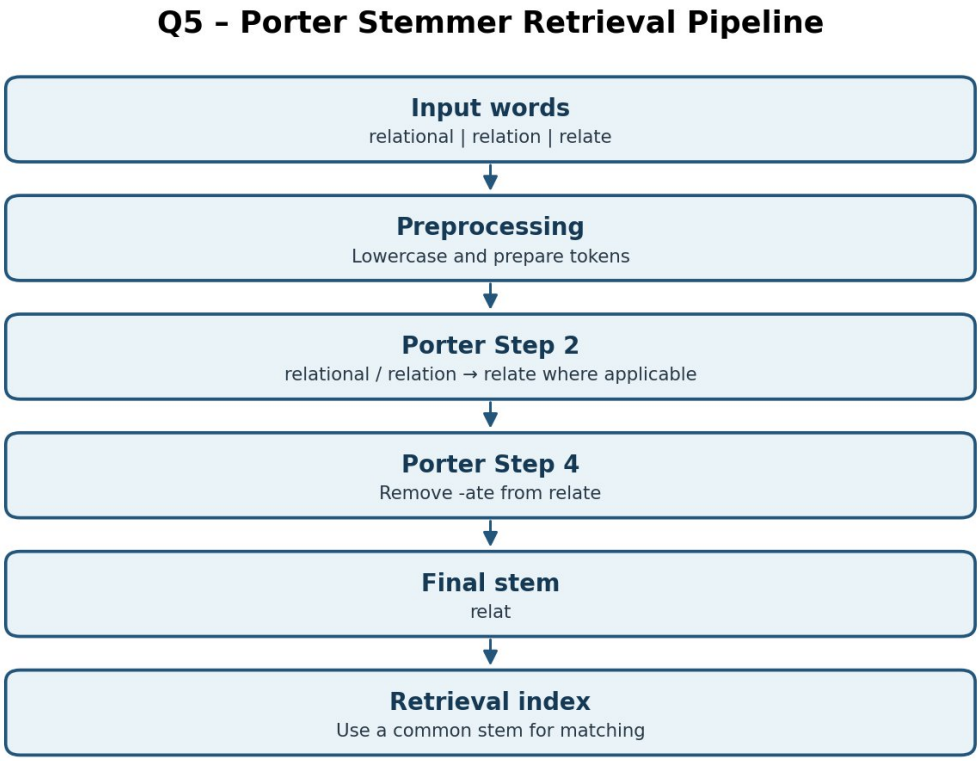
Objectives

- Apply the Porter stemming algorithm to all three words.
- Show the suffix rule applied at each stage.
- Record intermediate forms after each rule.
- Obtain the final stem for every input.
- Use the common stem for document retrieval.

Modules

1. Input and lowercase preprocessing
2. Porter Step 2 derivational suffix reduction
3. Porter Step 4 suffix removal
4. Final stem validation
5. Common retrieval-key generation

System architecture



Morphological and stemming breakdown

Word	Morphological view	Porter rule sequence	Intermediate forms	Final stem
relational	relate + -ion + -al (historical/derivational structure)	Step 2: -ational → -ate; Step 4: -ate → ∅	relational → relate → relat	relat
relation	relate + -ion (related noun form)	Step 2: -ation → -ate; Step 4: -ate → ∅	relation → relate → relat	relat
relate	Base lexical form for this set	Step 4: -ate → ∅	relate → relat	relat

Porter stemmer note: the stem relat is an algorithmic stem. It should not be interpreted as a complete English word or as a full morphological root.

Intermediate stemming steps

Input	Stage	Rule applied	Output after stage
relational	Initial	Original token	relational
relational	Porter Step 2	ational → ate	relate
relational	Porter Step 4	ate → empty	relat
relation	Initial	Original token	relation
relation	Porter Step 2	ation → ate	relate
relation	Porter Step 4	ate → empty	relat
relate	Initial	Original token	relate
relate	Porter Step 4	ate → empty	relat

Step-by-step algorithm

- Read each input token and convert it to lowercase.
- Create a PorterStemmer object using the NLTK library.
- For relational, apply the Step 2 mapping -ational → -ate, producing relate.
- For relation, apply the Step 2 mapping -ation → -ate, producing relate.
- For relate, no Step 2 change is needed.
- Apply Step 4 to remove the suffix -ate from each intermediate form, producing relat.
- Run the PorterStemmer implementation to verify the final stem.
- Display the original word, rule trace, intermediate forms and final stem.

Python program

```
from nltk.stem import PorterStemmer

words = ["relational", "relation", "relate"]
stemmer = PorterStemmer()

traces = {
    "relational": ["relational", "relate", "relat"],
    "relation": ["relation", "relate", "relat"],
    "relate": ["relate", "relat"]
}

rules = {
    "relational": ["Step 2: ational -> ate", "Step 4: ate -> empty"],
    "relation": ["Step 2: ation -> ate", "Step 4: ate -> empty"],
    "relate": ["Step 4: ate -> empty"]
}

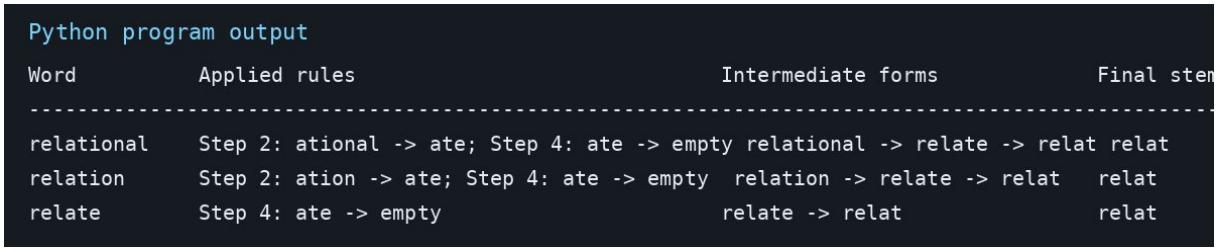
print(f"{'Word':<14}{'Applied rules':<42}{'Intermediate forms':<30}{'Final stem'}")
print("-" * 105)

for word in words:
    final_stem = stemmer.stem(word)
```

```
    rule_text = "; ".join(rules[word])
    form_text = " -> ".join(traces[word])
    print(f"{word:<14}{rule_text:<42}{form_text:<30}{final_stem}")
```

Installation command: pip install nltk

Sample output screenshot



Test cases

Input	Expected intermediate form(s)	Expected Porter stem	Status
relational	relational → relate → relat	relat	Pass
relation	relation → relate → relat	relat	Pass
relate	relate → relat	relat	Pass
RELATIONAL	Lowercase → relational → relate → relat	relat	Pass with lowercase preprocessing

Conclusion

The Porter Stemmer reduces relational, relation and relate to the common stem relat. Showing intermediate forms makes the ordered suffix processing clear. The common stem can be stored as an index key so documents containing related word forms can be retrieved together.